

UNIVERSITY OF CLAUDE · FREE RESOURCE MAP

Resource Map

CSCE 606 Software Engineering — module-by-module mapping onto free university courseware

HOW TO USE THIS

This is your lecture hall. For each module: watch or read the mapped external material first, then work the module slides and notes, which are written to assume you have seen it.

Where a specific lecture number is given, that is the one that covers this module. Where a book chapter is given, read that chapter, not the whole book.

Everything here is free. Resources remain under their own licences — link to them, do not redistribute them.

Primary sources for the whole course

Resource	Role in this course
Software Engineering at Google (free online)	The best single book on the subject, free from the authors. Its framing — engineering is programming integrated over time — is this course's framing.
MIT 6.031 Software Construction (free, complete)	Excellent on specification, invariants, and testing. The readings are unusually precise.
Berkeley CS169 — Engineering Software as a Service (edX, free audit)	A full project-based course, with the SaaS context that most software now lives in.
Martin Fowler — refactoring.com and martinfowler.com	The reference for refactoring, continuous integration, and architecture patterns. Catalogue-style and genuinely useful.
Google Testing Blog and the Test Sizes taxonomy	Practical testing writing from people running it at scale.
Accelerate / DORA research (summaries free)	The one body of empirical evidence about software delivery practices. Read it before believing anything else in this field.

Module-by-module map

Module 01 — Why Software Engineering Exists

Resource	What it covers for this module
Software Engineering at Google — Chapters 1–3	'Programming over time', the distinction from programming, and the Beyoncé rule. This course's framing comes from here.
Fred Brooks — No Silver Bullet (1986, widely available free)	Essential and accidental complexity. Forty years old and still the sharpest piece of analysis in the field.
Conway — How Do Committees Invent? (1968, free)	The original paper. Short, and the argument is better than the slogan.
DORA — State of DevOps research	The most substantial empirical evidence base in software delivery. Read the summaries before accepting any claim about process.

Module 02 — Requirements and Specification

Resource	What it covers for this module
MIT 6.031 — Specifications, Designing Specifications	The most precise free treatment of preconditions, postconditions, and specification strength. Read both readings.
Software Engineering at Google — Chapter 1 (and the 'what do we mean' framing)	On requirements that are discovered rather than stated.
Newcombe et al. — How Amazon Web Services Uses Formal Methods (free)	An honest industrial account of where formal specification pays and where it does not.
Michael Nygard — Documenting Architecture Decisions	The ADR format. Two pages, and widely adopted for good reason.

Module 03 — Modularity: Coupling and Cohesion

Resource	What it covers for this module
Parnas — On the Criteria To Be Used in Decomposing Systems into Modules (1972, free)	Ten pages, fifty years old, and still the best thing written on this. Read it in full.
Ousterhout — A Philosophy of Software Design	Deep modules, classitis, and complexity as a measurable property. Short and opinionated; the lecture video is free.
MIT 6.031 — Abstract Data Types, Abstraction Functions	The formal side: representation invariants and what an abstraction boundary actually guarantees.
Martin Fowler — Refactoring catalogue (code smells)	Shotgun surgery, feature envy, and the rest, with the corresponding refactorings.

Module 04 — Architecture

Resource	What it covers for this module
Martin Fowler — Software Architecture Guide	Including 'Who needs an architect?' and the microservices material, which is unusually balanced for the topic.
Fowler — MonolithFirst and Microservice Premium	The argument for §3's default, from someone who advocates microservices where they fit.
Alistair Cockburn — Hexagonal Architecture	The original description. Shorter and clearer than most of what has been written about it since.
Michael Nygard — Release It!	Stability patterns and failure modes in production systems. The chapter on failure cascades is worth the book.

Module 05 — Testing

Resource	What it covers for this module
Software Engineering at Google — Chapters 11–14 (Testing Overview, Unit Testing, Test Doubles, Larger Testing)	The test-size taxonomy, the argument against mock-heavy testing, and the honest discussion of what large tests are for.
MIT 6.031 — Testing	Partition-based test design and systematic case selection, which this module leaves to Module 06.
Google Testing Blog — flaky tests and test sizes	Practical writing from people operating very large test suites.
Fucci et al. — An External Replication on the Effects of Test-driven Development (free)	One of the more careful empirical studies of TDD. Read it before accepting strong claims in either direction.

Module 06 — Test Design

Resource	What it covers for this module
MIT 6.031 — Testing (partitioning and boundary analysis)	The most systematic free treatment of choosing test cases.
Hypothesis documentation — 'What is property-based testing?'	Excellent explanation plus a usable library. The equivalents in other languages (QuickCheck, fast-check, proptest) share the ideas.
Martin Fowler — Mocks Aren't Stubs	The taxonomy of doubles and the argument about classical versus mockist testing. Still the clearest statement of both positions.
Michael Feathers — Working Effectively with Legacy Code	Characterisation tests and seam-finding. The standard reference for testing code that was not designed to be tested.

Module 07 — Refactoring and Technical Debt

Resource	What it covers for this module
Martin Fowler — Refactoring catalogue	Every refactoring with its mechanics. The reference to keep open while working.
Fowler — Technical Debt Quadrant	The deliberate/inadvertent and prudent/reckless distinction of §2.
Ward Cunningham — Debt Metaphor (free video)	Cunningham explaining what he actually meant, and correcting the common misuse. Five minutes.
Joel Spolsky — Things You Should Never Do, Part I	The canonical argument against rewrites, with the Netscape case. Twenty years old and still correct.

Module 08 — Version Control and Collaboration

Resource	What it covers for this module
Pro Git (free book)	Chapter 10 ('Git Internals') is the one that makes everything else make sense. Read it before the command chapters.
Git from the Bottom Up (free)	Builds the model from the object store upward. Short, and the best explanation of why rebase works the way it does.
Conventional Commits	A structured message format that enables automated changelogs and version bumping. Useful when adopted consistently.
DORA — trunk-based development	The evidence behind §4's claim about branch lifetime.

Module 09 — Code Review

Resource	What it covers for this module
Google's Code Review Developer Guide (free)	The best free material on this, from both the reviewer's and the author's side. The 'approve when it improves things' standard comes from here.
Software Engineering at Google — Chapter 9 (Code Review)	Why they review, what they found it actually buys, and the data behind it.
SmartBear — Best Kept Secrets of Peer Code Review (free summary)	The source of the frequently cited size and duration findings.
Conventional Comments	A light convention for labelling comments by type and severity. Adopting it removes most review friction for free.

Module 10 — Continuous Integration and Delivery

Resource	What it covers for this module
Martin Fowler — Continuous Integration	The article that defined the practice, and still the clearest statement that it is a practice rather than a tool.

Resource	What it covers for this module
Fowler — Feature Toggles	The taxonomy of §3 and the management discipline that keeps flags from becoming debt.
DORA — research and the four key metrics	The evidence base. Read the capability pages, not just the metric definitions.
Humble & Farley — Continuous Delivery	The standard reference for pipelines, deployment strategies, and database migration under continuous delivery.

Module 11 — Observability

Resource	What it covers for this module
Google — Site Reliability Engineering (free online)	SLOs, error budgets, alerting philosophy, and incident response. The standard reference, and free.
Charity Majors et al. — Observability Engineering (sample chapters free)	The monitoring-versus-observability distinction and the argument for high-cardinality data.
OpenTelemetry documentation	The vendor-neutral standard for traces, metrics, and logs. Instrument with this rather than a proprietary agent.
Gil Tene — How NOT to Measure Latency (free video)	Why averages mislead, why percentile averaging is invalid, and what coordinated omission does to your measurements. Essential and entertaining.

Module 12 — Process

Resource	What it covers for this module
Royce — Managing the Development of Large Software Systems (1970, free)	The waterfall paper. Read sections 3 onward, where he argues against the model he had just drawn.
The Agile Manifesto and its twelve principles	Four sentences and twelve principles. Takes five minutes and corrects a great deal.
Forsgren, Humble & Kim — Accelerate / DORA	The empirical research. Read the capability pages rather than only the four metrics.
Google re:Work — Project Aristotle	The psychological safety finding, with the methodology.

Module 13 — Maintenance and the Long Life of Code

Resource	What it covers for this module
Software Engineering at Google — Chapters 1, 10, 21, 22 (time, documentation, dependencies, large-scale change)	Hyrum's law, dependency management at scale, and how to make a change across millions of lines. The deprecation material is particularly good.
Diátaxis documentation framework	The four-kind taxonomy of §2. Short, practical, and it immediately explains why most project documentation fails.

Resource	What it covers for this module
Lehman — Programs, Life Cycles, and Laws of Software Evolution (free summaries)	The laws, with the empirical work behind them.
Michael Feathers — Working Effectively with Legacy Code	What to do when you inherit a system that fails the handover test.